

AI Freelancer Quality Scoring Platform

1. Project Overview

The AI Freelancer Quality Scoring Platform is a web-based application designed to evaluate and recommend freelancers based on project performance, client feedback, profile information, and quality scores. The platform helps clients identify reliable freelancers through AI-enhanced quality scoring, freelancer rankings, and recommendation mechanisms.

The main goal of the system is to improve transparency and trust in freelance project management by analyzing freelancer performance using project tracking, sentiment analysis of client feedback, and quality scoring techniques.

The platform supports two types of users:

- **Freelancers:** Manage profiles, apply for projects, track scores and rankings, and view client reviews.
- **Clients:** Post projects, manage project progress, evaluate freelancers, view recommendations and rankings, and submit reviews.

Technology Stack: ASP.NET Core (C#), Angular (TypeScript), SQLite, ML.NET

2. Problem Statement

Freelance platforms often lack reliable methods to accurately evaluate freelancer performance. Traditional rating systems may not fully reflect work quality, timely delivery, and client satisfaction, making it difficult for clients to identify trustworthy freelancers.

The AI Freelancer Quality Scoring Platform addresses this issue by analyzing project performance, client feedback through sentiment analysis, and freelancer profile information to generate transparent quality scores, freelancer rankings, and project-based recommendations.

3. Features (Functional Scope)

User Authentication

- User registration and login
- JWT-based authentication
- Role-based authorization

Project Tracking

- Project creation and assignment
- Deadline tracking
- Project status updates
- Completion monitoring

Feedback Analysis

- Client review collection
- Sentiment analysis of feedback

Quality Scoring

- AI-based sentiment scoring
- Performance-based score updates
- Multi-factor evaluation system

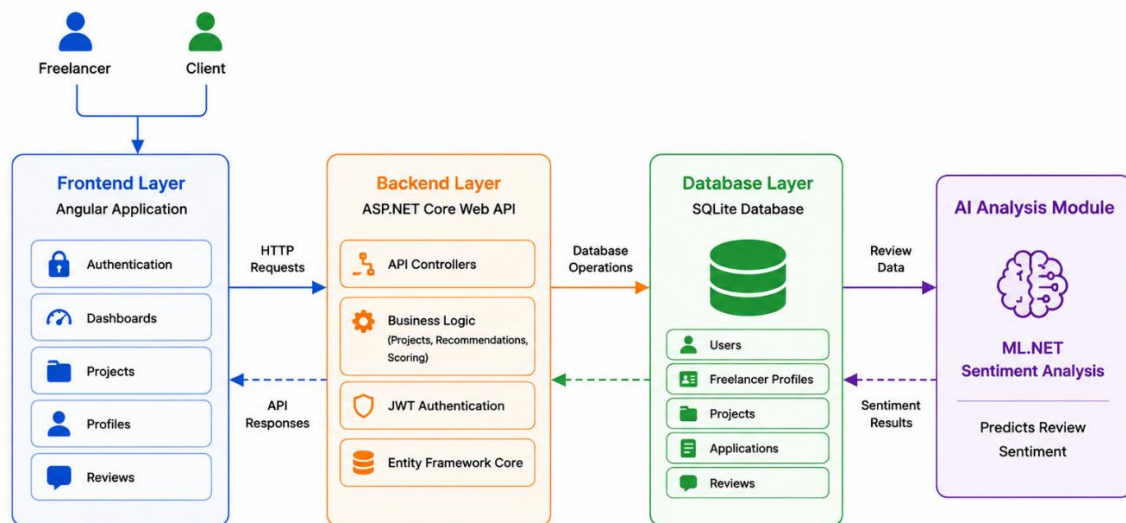
Freelancer Ranking

- Ranking freelancers based on scores
- Displaying top-performing freelancers
- Dynamic ranking updates

Freelancer Recommendation

- Skill-based freelancer matching
- Match percentage calculation
- Recommended freelancers for projects

4. System Architecture



5. Design

The AI Freelancer Quality Scoring Platform is designed using a modular client-server architecture that separates the frontend, backend, database, and AI processing components.

Frontend Design

The frontend is developed using Angular and TypeScript to provide a responsive and interactive user interface. The application uses component-based architecture, where each feature is divided into reusable standalone components.

Frontend Structure

- Authentication Components
 - Login
 - Registration
- Dashboard Components
 - Freelancer Dashboard
 - Client Dashboard
- Project Components
 - Project Creation
 - Project Tracking

- Project Details
- Ranking Components
 - Freelancer Rankings
 - Score Display
- Profile Components
 - Freelancer Profile
 - Profile Editing

Frontend Implementation

- Angular routing is used for navigation between modules.
- Route guards protect restricted pages based on user roles.
- HTTP Client communicates with backend REST APIs.
- JWT tokens are stored locally for authenticated sessions.

Backend Design

The backend is implemented using ASP.NET Core Web API and follows a layered architecture for better organization of business logic and database operations.

Backend Layers

API Controller Layer

Handles incoming HTTP requests and sends responses to the frontend.

Logic Layer

Processes application logic such as:

- Score calculation
- Ranking updates
- Project status handling
- Feedback processing

Data Access Layer

Manages communication with the SQLite database using Entity Framework Core.

Authentication Layer

Handles:

- User authentication
- Password encryption
- JWT token generation
- Role-based authorization

Database Design

SQLite is used as the relational database for storing application data.

Main Database Entities

- Users
- Projects
- Reviews
- FreelancerProfiles
- ProjectApplications

Scoring Design

The scoring module evaluates freelancers using multiple performance factors.

Parameters Considered

- Delivery punctuality
- Client ratings
- Feedback sentiment
- Number of completed projects

Scoring Process

1. Project and review data are collected.
2. Feedback text is analyzed using sentiment analysis.
3. Weighted scoring logic calculates overall quality score.
4. Rankings are updated dynamically based on scores.

6. Module Design

Authentication Module

- User registration
- Login validation
- JWT token generation
- Session handling

Freelancer Module

- Freelancer profile management
- Applying for projects
- Viewing scores and rankings
- Viewing client reviews

Client Module

- Posting projects
- Assigning freelancers
- Viewing recommendations
- Submitting reviews and ratings

Project Tracking Module

- Project creation
- Status tracking
- Deadline monitoring
- Completion management

Feedback Analysis Module

- Processing reviews
- ML.NET Sentiment analysis
- Feedback categorization

Quality Scoring Module

- Score generation

- Scoring Parameters
 - Timely project delivery
 - Client ratings
 - Feedback sentiment
 - Project completion rate

Ranking Module

- Ranking freelancers based on Overall Score
- Updating rankings dynamically
- Displaying top freelancers

Recommendation Module

- Suggest freelancers for projects
- Match percentage calculation

7. API Design

Authentication APIs

❖ Register User

- **Endpoint:**
POST /api/Auth/register
- **Input:**
 - Username
 - Email
 - Password
 - Role
- **Output:**
 - Success message
 - Registered user details

❖ Login User

- **Endpoint:**
POST /api/Auth/login
- **Input:**

- Email
- Password
- **Output:**
 - JWT token
 - User role
 - User ID
 - Username

Project APIs

❖ Create Project

- **Endpoint:**
POST /api/Projects
- **Input:**
 - Title
 - Description
 - Requirements
 - Budget
 - Deadline
- **Output:**
 - Success message
 - Created project

❖ Get My Projects

- **Endpoint:**
GET /api/Projects/my-projects
- **Output:**
 - Projects posted by the logged-in client

❖ Get All Projects

- **Endpoint:**
GET /api/Projects
- **Output:**
 - List of available projects

❖ Get Assigned Projects

- **Endpoint:**
GET /api/Projects/assigned

- **Output:**
 - Assigned projects for freelancer

❖ **Submit Review**

- **Endpoint:**
POST /api/Projects/review
- **Input:**
 - ProjectId
 - FreelancerId
 - Rating
 - Feedback
- **Output:**
 - Review submitted successfully
 - Updated freelancer score

❖ **Get Recommended Freelancers**

- **Endpoint:**
GET /api/Projects/{projectId}/recommendations
- **Output:**
 - Recommended freelancers
 - Match percentage

Project Application APIs

❖ **Apply for Project**

- **Endpoint:**
POST /api/ProjectApplications/{projectId}
- **Output:**
 - Application submitted successfully

❖ **View Applicants**

- **Endpoint:**
GET /api/ProjectApplications/project/{projectId}
- **Output:**
 - List of applicants

❖ **Accept Application**

- **Endpoint:**
PUT /api/ProjectApplications/accept/{applicationId}
- **Output:**
 - Freelancer assigned to project

Rankings API

❖ Get Rankings

- **Endpoint:**
GET /api/Rankings
- **Output:**
 - Top freelancers with their
 - Overall score
 - Rating
 - Completed projects count

Reviews API

❖ Get My Reviews

- **Endpoint:**
GET /api/Reviews/my-reviews
- **Output:**
 - Ratings
 - Feedback

8. Data Flow

Registration Flow

1. User submits registration form.
2. Frontend sends data to backend API.
3. Backend validates data.
4. User details are stored in SQLite database.

Login Flow

1. User enters login credentials.
2. Backend validates credentials.
3. JWT token is generated.
4. Token is returned to frontend.
5. User gains authorized access.

Project Tracking Flow

1. Client creates a project.
2. Freelancer works on the project.
3. Project progress and completion are tracked.
4. Completion details are stored in database.

Feedback & Scoring Flow

1. Client submits review and rating.
2. Feedback is analyzed by AI model.
3. Quality score is updated.
4. Rankings are recalculated.
5. Updated scores are displayed.

Recommendation Flow

1. Client creates a project.
2. System compares project requirements with freelancer profiles.
3. Match percentage is calculated.
4. Recommended freelancers are displayed.

9. Error Handling

Frontend Error Handling

- API error messages
- Route protection

Backend Error Handling

- Proper HTTP status codes
- Validation checks

Database Error Handling

- Duplicate entry prevention
- Connection failure handling

Authentication Error Handling

- Unauthorized access prevention

10. Assumptions & Limitations

Assumptions

- Users provide genuine information.
- Clients submit honest feedback.
- Internet connection is available.

- The sentiment model is trained using review data.

Limitations

- AI scoring may not always be perfectly accurate.
- Sentiment analysis may misinterpret some reviews.
- New freelancers may initially have lower visibility.